Peng Bohao - Project Portfolio Page

Overview

JobSeek is a command-line desktop application (CLI) designed for undergraduate students and job seekers to track and manage their internship and graduate-role applications, monitor upcoming deadlines, and organize their recruitment pipeline.

Summary of Contributions

Code contributed

RepoSense Dashboard

Enhancements implemented

- **Foundational Architecture & Code Skeleton** (#13, #14):
  - *What it does*: Bootstrapped initial JobSeek codebase from scratch, establishing core domain models (`Application` , `ApplicationList`), command pattern hierarchy (`Command`), console interface (`Ui`), and persistence (`Storage`) with Java 25.
  - *Justification & Highlights*: Established modular package structures early so team members could develop concurrently without merge conflicts. Decoupled business logic from I/O and encapsulated list operations with index bounds protection.
- **One-line Typed Command Parser & Prefix Sanitization** (#50, #78):
  - *What it does*: Built `CommandParser` enabling single-line typed commands (`add c/Google r/SWE d/2026-11-30` , `find ...` , `undo`) alongside the numbered menu.
  - *Justification & Highlights*: Eliminates multi-step prompts for power users. Implemented regex prefix tokenization requiring whitespace, preventing false splits on slashes within text (`AT&T/Intel` , `C/C++`), and allowed clearing optional fields via prefix (`edit 1 d/ -`). Backed by 24 unit tests.
- **Multi-criteria Filter Engine (`find`)** (#51):
  - *What it does*: Filters applications across company (`c/`), role (`r/`), stage (`s/`), deadline range (`a/` , `b/`), and status (`m/`) simultaneously.
  - *Justification & Highlights*: Applied Strategy pattern (`ApplicationFilters`) composing `Predicate<Application>` lambdas joined via logical `AND` . Extensible for new filters without modifying `FindCommand` (OCP). Results retain master 1-based indexes for direct use with `edit` , `delete` , or `view` .
- **Reversible Undo Architecture & Robust Mutation Handling** (#52, #77, #83):
  - *What it does*: Added deletion confirmation prompts and `undo` command to iteratively revert mutating operations (`add` , `delete` , `edit` , `status` , `mark` , `unmark`).
  - *Justification & Highlights*: Protects against accidental data loss. Architected `UndoableCommand` and stack-based `CommandHistory` storing inverse state deltas. Prevented no-op undo entries for unchanged edits, supported cancellation (`cancel`), and handled abrupt EOF safely without corrupting history. Backed by 18 unit tests.
- **Storage Reliability, Atomic Saves & Corruption Recovery** (#45, #47, #76, #82):
  - Replaced destructive direct writes with temporary-file atomic replacement to eliminate truncated files on crash. Added UTF-8 BOM handling, automatic backups before skipping corrupted records, and persisted recruitment stages across restarts.
- **Model Encapsulation & Defensive Invariants** (#84):
  - Hardened `Application` to reject blank companies/roles using Unicode-aware validation (`String.isBlank()`). Fully encapsulated `ApplicationList` via unmodifiable collection views, defensive copies, and non-null guarantees (`NullPointerException`).
- **Application Details, Listing, and Status** (#32, #34):
  - Implemented `view INDEX` details card, filtered `list [completed|all]` , `mark` / `unmark` completion toggles, and `today` deadline dashboard.

Contributions to the User Guide

- **Authored sections**: One-line typed command syntax and prefix rules, Finding with combined filters, Undoing changes and delete confirmation, Today dashboard, Listing applications, Viewing application details, Marking/unmarking applications, and Command Summary quick reference.
- **Updated sections & audit** (#53, #78, #80): Adding applications (deadlines), Deleting applications (confirmation), Data persistence (status saving, corrupted-line recovery), clearing optional fields, and prompt cancellation.

Contributions to the Developer Guide

- **Authored design sections**: Typed command parser, Multi-criteria filter engine, Delete confirmation and undo architecture, Today dashboard, View command, Filtered listing, and Mark/unmark features (with design considerations and alternatives).
- **Documented architectural robustness** (#82, #84): Storage atomic file replacement guarantees and non-atomic fallback limitations, model encapsulation, and non-null domain invariants.
- **UML diagram standardization & audit** (#89): Standardized 25 diagrams and `.mmd` sources across the DG to conform strictly with CS2113 UML guidelines: applied `<<class>>` static lifelines, `<<create>>` arrows, `{abstract}` notation, aggregation/composition diamonds, and rounded activity nodes.
- **Documentation restructuring & sections filled** (#53): Fixed section corruption, authored product scope, non-functional requirements (NFRs), glossary, user stories, and manual testing procedures.

Contributions to team-based tasks

- Set up initial codebase skeleton (`Application` , `Command` , `Ui` , `Storage` , `ApplicationList` , `Main`) and configured `build.gradle` for Java 25 (#13).
- Engineered diagram rendering build script (`docs/diagrams/render-diagrams.cjs`) automating headless Mermaid-to-PNG generation and SVG post-processing for consistent, high-resolution DG documentation (#89).
- Isolated test infrastructure so unit and integration test executions save to temporary files instead of overwriting user data in `data/applications.txt` (#79).
- Updated project branding across README, organized help text into typed-command and menu lists (#80), and configured team member portfolio links in `AboutUs.md` .
- Opened, tracked, and resolved GitHub issues for features and bug fixes (e.g., #27, #46), and reviewed all documentation against implementation.

Review/mentoring contributions

- **Internal Team Reviews & Mentoring**: Reviewed and merged teammates' PRs (e.g., PR #1), assisted in resolving git merge conflicts across feature branches, aligned command-model interfaces, and audited PRs for Checkstyle and test coverage.
- **Cross-Team DG & UML Review**: Provided 5 detailed review comments on the Developer Guide and UML diagrams of Team TutorLedger (CS2113-W12-2) (PR #5 Review), advising on standard PlantUML stereotypes, class compartment notation, instance lifeline headers, dependency arrows, and activity diagram decision nodes.

Contributions beyond the project team

- **Cross-Team Bug Reporting (V2.0 PE Dry Run)**: Reported high-severity defects across external teams:
  - **Team CakeQueue (W12-1)** (#39): Identified silent data loss when saving after encountering corrupted records, and prefix delimiter parser collision on input strings ending with slashes.
  - **Team BMS (T11-1)** (#116): Identified unhandled `ArithmeticException` crash during scientific-notation price parsing, and silent integer overflow into negative quantities upon stock increments.